

# **Modelleren van computers en berekeningen: Turing Machines**

1001WETALC

## Deterministische Turing machines

Algoritmen & beslissers

Tijdscomplexiteit

Asymptotische analyse

Beslissingsproblemen

Deterministische Turing machines met meerdere tapes

Niet-Deterministische Turing machines

In eerdere cursussen (talen en automaten, machines en berekenbaarheid):

## **Eindige automaten**

- ▶ modelleren van machines met beperkt geheugen.
- ▶ reguliere talen.

## **Pushdown automaten**

- ▶ modelleren van machines met oneindig geheugen.
- ▶ geheugen is een “stack” (laatste toevoeging wordt eerst gebruikt).
- ▶ contextvrije grammatica's.

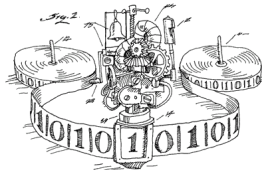
Deze zijn echter **te restrictief** om algemene computers te modelleren.

Daarom: **Turing machines** (TM) (zie ook cursus: machines en berekenbaarheid).

Waarom zijn deze een goede formalisering?

- ▶ Alle **bekende algoritmen** kunnen worden geïmplementeerd door een Turing machine.
- ▶ Alle **programma's** van een Turing machine kunnen worden geïmplementeerd in een programmeertaal dat uitgevoerd kan worden op een computer.
- ▶ Elke andere **bestaande formalisering kan herleid** worden tot die van een Turing machine.
  - ▶ en dit op een **efficiënte** manier.

**Church-Turing thesis:** **Algoritme = Turing machine.**



Bestaat uit:

- ▶ Een **eindige controle** die de machine in verschillende toestanden kan brengen;
- ▶ Een **oneindige tape** waarvan gelezen en waarop geschreven kan worden;
- ▶ Een **lees/schrijfkop** die kan bewegen over de tape; en
- ▶ Een **aanvaardings/verwerping**s toestand dewelke ons zegt of een input al dan niet aanvaard wordt.

Merk op dat zo'n machine in het algemeen niet noodzakelijk moet stoppen.

**Alfabet  $\Sigma$ :** een eindige verzameling van symbolen.

- ▶ Voorbeeld:  $\Sigma = \{0, 1\}$ .

**Woord  $w$  over  $\Sigma$ :** een eindige reeks van symbolen in  $\Sigma$ .

- ▶ Voorbeeld:  $w = 01101$ .

**$\Sigma^*$ :** De verzameling van alle woorden over  $\Sigma$ .

Een **taal  $L$  (over  $\Sigma$ )**: een verzameling van woorden over  $\Sigma$ .

- ▶ Voorbeeld  $L = \{0^n 1^n \mid n \in \mathbb{N}\}$ .

## Definition

Een **deterministische Turing machine**  $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$  bestaat uit:

- ▶  $Q$  een eindige verzameling van **toestanden** (states);
- ▶  $\Sigma$  het **input alfabet**, met  $\vdash, \sqcup \notin \Sigma$ ;
- ▶  $\Gamma$  het **tape alfabet**, met  $\Sigma \cup \{\vdash, \sqcup\} \subseteq \Gamma$ ;
- ▶  $q_0 \in Q$  de **begintoestand** (initial state);
- ▶  $q_{accept} \in Q$  de **aanvaardingstoestand** (accepting state);
- ▶  $q_{reject} \in Q$  de **verwerpingstoestand** (rejecting state) ( $q_{accept} \neq q_{reject}$ ); en
- ▶  $\delta$  een partiële functie:  $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \square, \rightarrow\}$ ,  $\delta$  wordt ook de **transitiefunctie** genoemd.

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \square, \rightarrow\}$$

Gegeven een toestand  $q \in Q$  en tape symbool  $a \in \Gamma$ ,

- ▶  $\delta(q, a) = (q', b, \leftarrow)$ : schrijf  $b$  op huidige positie, gaan naar **links** op de tape, en ga over naar toestand  $q'$
- ▶  $\delta(q, a) = (q', b, \square)$ : schrijf  $b$  op huidige positie, **blijf staan**, en ga over naar toestand  $q'$
- ▶  $\delta(q, a) = (q', b, \rightarrow)$ : schrijf  $b$  op huidige positie, gaan naar **rechts** op de tape, en ga over naar toestand  $q'$



De tape van de Turing machine **loopt oneindig door langs de rechterkant**. We veronderstellen:

- ▶ het  $\vdash$  symbool markeert de positie 0 (meest linkse positie) op de tape.

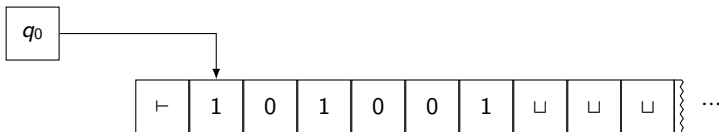
We veronderstellen ook dat:

- ▶ Als  $\delta(q, \vdash)$  gedefinieerd is, dan
  - ▶  $\delta(q, \vdash) = (q', \vdash, X)$ , met  $X \in \{\rightarrow, \square\}$ ;
  - ▶ we kunnen dus niet naar links bewegen of  $\vdash$  overschrijven.
- ▶ Als  $a \in \Gamma \setminus \{\vdash\}$  en  $\delta(q, a)$  gedefinieerd is, dan
  - ▶  $\delta(q, a) = (q', b, X)$ , met  $b \in \Gamma \setminus \{\vdash\}$ ;
  - ▶ we kunnen dus geen  $\vdash$  symbool schrijven op andere posities dan de beginpositie.

Dit maakt het gemakkelijk om terug naar de beginpositie te gaan.

Stel,  $\Sigma$  is het input alfabet en  $\Gamma$  is het tape alfabet

- ▶ Een input  $w \in \Sigma^*$  van lengte  $n$  staat initieel op de posities  $1, \dots, n$  van de tape
- ▶ De posities  $(n + 1, n + 2, \dots)$  bevatten het symbool  $\sqcup$  ("blank")
- ▶ De machine start steeds in de toestand  $q_0$  en met de lees/schrijfkop op positie 1 van de tape



De **uitvoering** (run) van de machine zorgt voor een (mogelijke) verandering in:

- ▶ de huidige toestand van de machine;
- ▶ de inhoud van de tape; en
- ▶ de huidige positie van de lees/schrijfkop.

Een welbepaalde combinatie van deze drie componenten wordt een **configuratie** van de machine genoemd.

# Berekening door een Turing machine: Representatie van configuraties

---

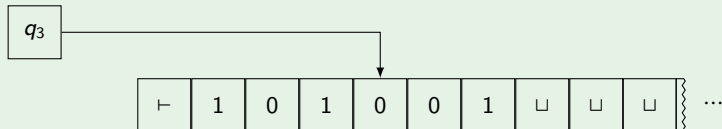
Veronderstel dat:

- ▶ de huidige **toestand** van de machine is  $q$ ;
- ▶ de **inhoud** van de tape is van de vorm  $uv$ , met  $u$  en  $v$  woorden over  $\Gamma$  (en alle symbolen op de tape op posities na de laatste letter van  $v$  zijn gelijk aan  $\sqcup$ ); en
- ▶ de **lees/schrijfkop** van de machine staat op de eerste letter van  $v$ .

We **representeren deze configuratie** dan met  $C \models uv$ .

## Voorbeeld

Stel, we hebben de volgende situatie:



Dan, deze configuratie wordt beschreven door middel van  $u = 101$  and  $v = 001$ .

En dus, dit resulteert in de configuratie  $C = \vdash 101q_3001$ .

Gedurende een “run” van TM, zeggen we dat een configuratie  $C_1$  **leidt** tot een configuratie  $C_2$  wanneer hetvolgende geldt:

Voor  $a, b, c \in \Gamma$ , en  $u, v \in \Gamma^*$  en  $q_i, q_j \in Q$ ;

$uaq_i bv$  leidt tot  $uq_j acv$ , wanneer  $\delta(q_i, b) = (q_j, c, \leftarrow)$ ;

$uaq_i bv$  leidt tot  $uacq_j v$ , wanneer  $\delta(q_i, b) = (q_j, c, \rightarrow)$ ; of

$uaq_i bv$  leidt to  $uaq_j cv$ , wanneer  $\delta(q_i, b) = (q_j, c, \square)$ .

De **startconfiguratie** op input  $w$  is altijd  $\vdash q_0 w$

In de **aanvaardingsconfiguratie** de toestand is altijd  $\vdash q_{accept}$ .

In de **verwerpingsconfiguratie** de toestand is altijd  $\vdash q_{reject}$ .

De toestanden  $q_{accept}$  en  $q_{reject}$  zijn zogenaamde **stoptoestanden** (halting states).

Wanneer de machine zich in een stoptoestand bevindt kunnen geen verdere configuraties worden bekomen.

## Voorbeeld

We maken een Turing machine die **nakijkt of het aantal nullen in een woord op de tape even is**:

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$  met

- ▶  $Q = \{q_0, q_1, q_{accept}, q_{reject}\};$
- ▶  $\Sigma = \{0, 1\};$
- ▶  $\Gamma = \{0, 1, \vdash, \sqcup\};$  en
- ▶  $\delta$  is gedefinieerd als volgt:

$$\delta(q_0, 0) = (q_1, \sqcup, \rightarrow)$$

$$\delta(q_0, 1) = (q_0, \sqcup, \rightarrow)$$

$$\delta(q_1, 0) = (q_0, \sqcup, \rightarrow)$$

$$\delta(q_1, 1) = (q_1, \sqcup, \rightarrow)$$

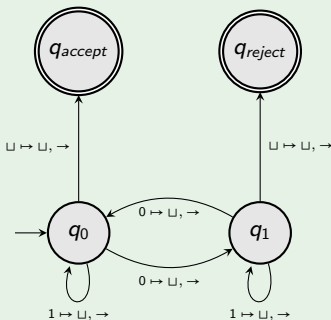
$$\delta(q_0, \sqcup) = (q_{accept}, \sqcup, \rightarrow)$$

$$\delta(q_1, \sqcup) = (q_{reject}, \sqcup, \rightarrow).$$



## Voorbeeld

$$\begin{aligned}\delta(q_0, 0) &= (q_1, \sqcup, \rightarrow) \\ \delta(q_0, 1) &= (q_0, \sqcup, \rightarrow) \\ \delta(q_1, 0) &= (q_0, \sqcup, \rightarrow) \\ \delta(q_1, 1) &= (q_1, \sqcup, \rightarrow) \\ \delta(q_0, \sqcup) &= (q_{accept}, \sqcup, \rightarrow) \\ \delta(q_1, \sqcup) &= (q_{reject}, \sqcup, \rightarrow)\end{aligned}$$

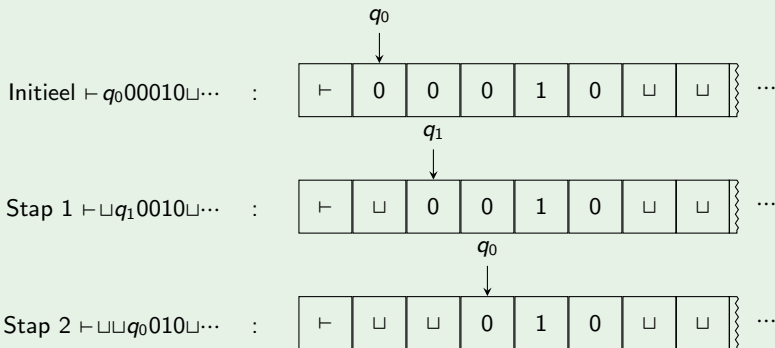


(We hebben hier de transities weggelaten die terug naar  $\vdash$  gaan om zo  $\vdash q_{accept}$  of  $\vdash q_{reject}$  te verkrijgen als eindconfiguratie. We doen dat wel vaker in de voorbeelden.)

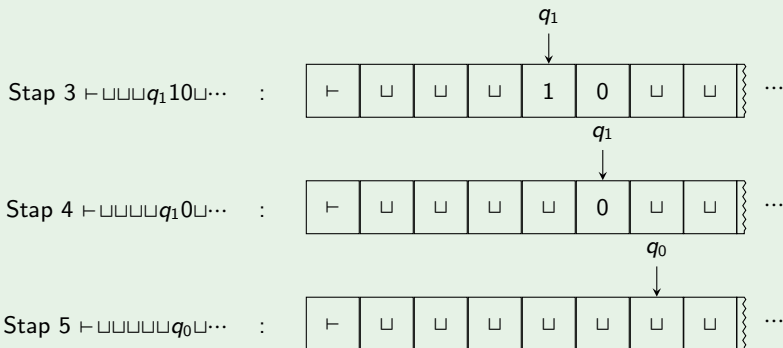
## Voorbeeld

Neem de TM  $M$  zoals daarnet beschreven.

Veronderstel  $w = 00010$ :



## Voorbeeld



In de volgende stap gaan we dan naar  $q_{accept}$ .

**Conclusie:** De machine **aanvaardt**  $w = 00010$

Zie ook: <https://turingmachinesimulator.com>

## Definition

Een Turing machine  $M$  **aanvaardt een input woord**  $w$  als er een reeks van configuraties  $C_1, C_2, \dots, C_k$  bestaat zodat:

1.  $C_1$  is de startconfiguratie van  $M$  op input  $w$ ;
2.  $C_i$  leidt tot  $C_{i+1}$ ; en
3.  $C_k$  is een aanvaardingsconfiguratie.

## Definition

De **taal aanvaard** door een Turing machine  $M$ :

$$L(M) = \{w \in \Sigma^* \mid M \text{ aanvaardt } w\}.$$

Een taal is **Turing-herkenbaar** (recursief opnoembaar) wanneer het aanvaard wordt door een Turing machine

## Example

$$A_{TM} = \{(M, w) \mid M \text{ is een TM en } M \text{ aanvaardt } w\}$$

is een Turing-herkenbare taal. De complement taal

$$\bar{A}_{TM} = \{(M, w) \mid M \text{ is een TM en } M \text{ aanvaardt } w \text{ niet}\}$$

is niet Turing-herkenbaar. (zie cursus: Machines en berekenbaarheid).

In het algemeen, een Turing machine kan op een input: *aanvaarden*, *verwerpen*, of **niet stoppen** (en loopt dus oneindig door).

Wanneer een machine niet stopt is dat **niet erg nuttig** om te beslissen of een woord in een taal zit.

Met andere woorden, dit levert **geen algoritme** op.

Deterministische Turing machines

**Algoritmen & beslissers**

Tijdscomplexiteit

Asymptotische analyse

Beslissingsproblemen

Deterministische Turing machines met meerdere tapes

Niet-Deterministische Turing machines

**Algoritme** = Turing machine dewelke stopt op **elke** input.

### Definition

Een Turing machine is een **beslisser** als deze stopt op elke input.

Een taal is **Turing-beslisbaar** (recursief) als deze aanvaard wordt door een beslisser

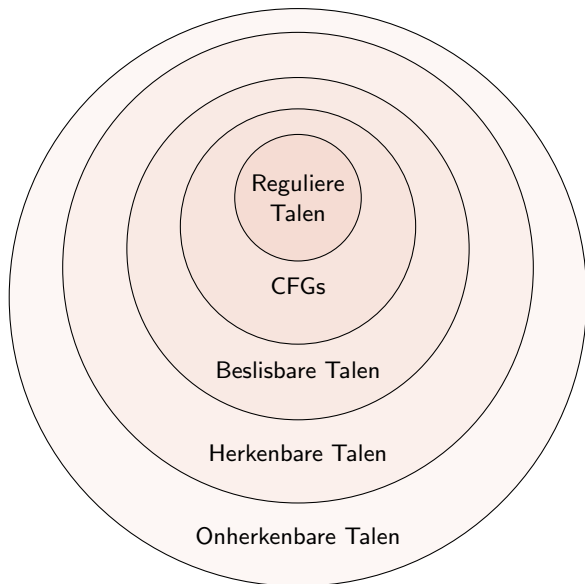
### Example

De machine  $M$  (even aantal nullen) is een beslisser voor:

$$L(M) = \{w \in \{0,1\}^* \mid w \text{ heeft een even aantal } 0\}$$

en  $L(M)$  is dus Turing-beslisbaar. Echter,  $A_{TM}$  is niet beslisbaar. (zie cursus: Machines en berekenbaarheid).

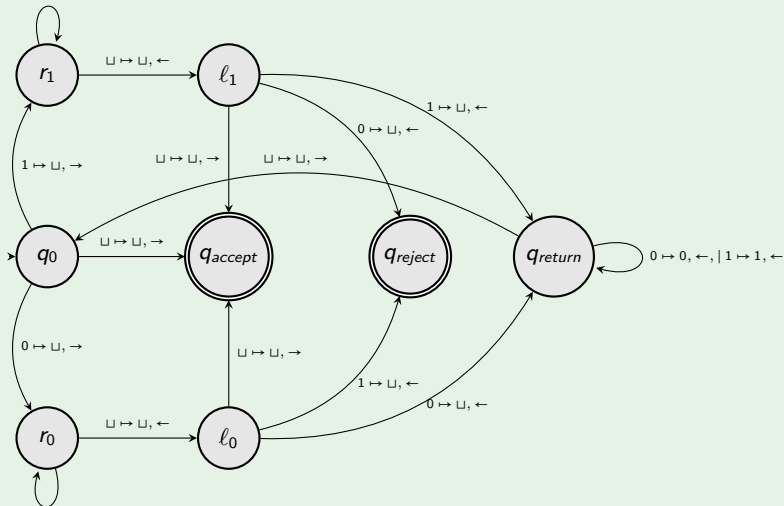
Vanaf nu, alle Turing machines die we beschouwen zijn beslissers!





## Voorbeeld

$1 \mapsto 1, \rightarrow \mid 0 \mapsto 0, \rightarrow$



$1 \mapsto 1, \rightarrow \mid 0 \mapsto 0, \rightarrow$

### Voorbeeld

$M_p$  = op input woord  $w$ :

1. Als  $w$  leeg is, aanvaard.
2. Zoniet, stel dat het eerste symbool 0 (resp. 1) is. Overschrijf met  $\sqcup$  en ga naar het einde van het woord.
3. Als het laatste symbool ook 0 (resp. 1) is, overschrijf met  $\sqcup$ .
4. Als er twee verschillende symbolen gelezen worden, verwerp de input, anders ga door.
5. Als links van de kop  $\sqcup$  staat, aanvaard, anders ga naar het eerste  $\sqcup$  symbool links van de kop.
6. Herhaal.

$$L(M_p) = \{w \in \{0,1\}^* \mid w \text{ is een palindroom}\}.$$

Deterministische Turing machines

Algoritmen & beslissers

**Tijdscomplexiteit**

Asymptotische analyse

Beslissingsproblemen

Deterministische Turing machines met meerdere tapes

Niet-Deterministische Turing machines

Kwantificeer de **computationale complexiteit** van een probleem

Meer bepaald: We willen meten hoeveel **computationale middelen nodig zijn** om een probleem op te lossen:

- ▶ tijd;
- ▶ Ruimte (geheugen);
- ▶ Randomness;
- ▶ ...

Eerst, wat is de tijdscomplexiteit van een algoritme?

Algoritme = TM die stopt op elke input

Hoe kunnen we nu de **tijd** meten die gebruikt wordt door het algoritme?

Voor een TM met alfabet  $\Sigma$ :

- ▶ **Stap van  $M$** : Het uitvoeren van **één instructie van de transitie functie** (van de ene configuratie naar een andere).
- ▶  **$time_M(w)$** : **Aantal stappen** dat  $M$  uitvoert op input  $w \in \Sigma^*$ .

## Definition

De “worst case” tijdscomplexiteit van een TM  $M$  is gedefinieerd als de functie  $t_M$ :

$$t_M(n) = \max\{ \text{time}_M(w) \mid w \in \Sigma^* \text{ en } |w| = n \}.$$

Hier,  $|w|$  is de lengte van de input.

### Voorbeeld

$M_p$  = Op input woord  $w$ :

1. Als  $w$  leeg is, aanvaardt. (**max 1 stap**)
2. Zoniet, stel het eerste symbool is 0 (resp. 1). Overschrijf met  $\sqcup$  en ga naar het einde van het woord. (**max  $n$  stappen**)
3. Als het laatste symbool ook 0 (resp. 1) is, overschrijf met  $\sqcup$ . (**max 1 stap**)
4. Als er twee verschillende symbolen gelezen worden, verwerp de input, anders ga door. (**max 1 stap**)
5. Als links van de kop  $\sqcup$  staat, aanvaard, anders ga naar het eerste  $\sqcup$  symbool links van de kop. (**max  $n$  stappen**)
6. Herhaal. (**max  $n/2$  keer**)

---

Totaal: maximaal  $\frac{n}{2}(2n + 2)$  **stappen**

- ▶ In complexiteitstheorie zijn we zelden geïnteresseerd in de exacte tijdscomplexiteit (precies aantal stappen).
- ▶ Eerder, we zijn geïnteresseerd in hoe tijdscomplexiteit groeit **wanneer input langer wordt**.
- ▶ We kijken daarom naar tijdscomplexiteit wanneer de input lengte naar oneindig gaat (**asymptotisch gedrag**).



Deterministische Turing machines

Algoritmen & beslissers

Tijdscomplexiteit

**Asymptotische analyse**

Beslissingsproblemen

Deterministische Turing machines met meerdere tapes

Niet-Deterministische Turing machines

Zie ook cursus: Gegevensabstractie en -structuren

## Definition

- ▶ **Big-O ( $\leq$ ) asymptotische bovengrens:**  $f(n) = \mathcal{O}(g(n))$  als:  
 $\exists c, n_0$  zodat  $\forall n \geq n_0, f(n) \leq c \cdot g(n)$
- ▶ **Big-Omega ( $\geq$ ) asymptotische benedengrens:**  $f(n) = \Omega(g(n))$  als:  
 $g(n) = \mathcal{O}(f(n))$ .

We gebruiken deze om de tijds- (en later plaats-) complexiteit te meten.

## Voorbeeld

$M_p$  = Op input woord  $w$ :

1. Als  $w$  leeg is, aanvaardt. ( $\mathcal{O}(1)$  **stappen**)
2. Zoniet, stel het eerste symbool is 0 (resp. 1). Overschrijf met  $\sqcup$  en ga naar het einde van het woord. ( $\mathcal{O}(n)$  **stappen**)
3. Als het laatste symbool ook 0 (resp. 1) is, overschrijf met  $\sqcup$ . ( $\mathcal{O}(1)$  **stappen**)
4. Als er twee verschillende symbolen gelezen worden, verwerp de input, anders ga door. ( $\mathcal{O}(1)$  **stappen**)
5. Als links van de kop  $\sqcup$  staat, aanvaard, anders ga naar het eerste  $\sqcup$  symbool links van de kop. ( $\mathcal{O}(n)$  **stappen**)
6. Herhaal. ( $\mathcal{O}(n)$  **keer**)

---

Totaal: maximaal  $\mathcal{O}(n) \cdot \mathcal{O}(n) = \mathcal{O}(n^2)$  **stappen**

## Definition

Een TM  $M$  loopt in **tijd**  $\mathcal{O}(t(n))$  als  $t_M(n)$  **van orde**  $\mathcal{O}(t(n))$  is.

## Voorbeeld

De palindroom TM  $M_p$  loopt in tijd  $\mathcal{O}(n^2)$ .

Deterministische Turing machines

Algoritmen & beslissers

Tijdscomplexiteit

Asymptotische analyse

**Beslissingsproblemen**

Deterministische Turing machines met meerdere tapes

Niet-Deterministische Turing machines

Herinner: We willen de computationele complexiteit weten van een probleem.

- ▶ **Beslissers** vormen onze klasse van algoritmen; en
- ▶ Gegeven een beslisser kunnen we de **tijdscomplexiteit** berekenen.

Wat voor soort **problemen** gaan we beschouwen?

## Definition

Het **beslissingsprobleem geassocieerd met een taal**  $L$ : Gegeven een woord  $w \in \Sigma^*$ , beslis (ja/nee) of  $w \in L$ .

## Voorbeeld

- ▶  $\text{PAL} = \{w \in \Sigma^* \mid w \text{ is een palindroom.}\}$ .
- ▶  $\text{EVEN} = \{w \in \Sigma^* \mid w \text{ bevat een even aantal nullen}\}$ .
- ▶ ...

We zien later in de cursus nog veel andere beslissingsproblemen.

## Definition

Een taal  $L$  **wordt beslist** door een TM  $M$  in **tijd**  $\mathcal{O}(t(n))$  als  $L = L(M)$  en  $t_M(n)$  is van orde  $\mathcal{O}(t(n))$ .

- De definitie is hetzelfde voor  $\Omega(t(n))$

## Definition

De **complexiteit** (tijd of andere) **van een taal**  $L$  is de complexiteit van het beslissingsprobleem geassocieerd met  $L$ .



Wanneer kunnen we zeggen dat (het beslissingsprobleem behorende bij)  $L$  efficiënt kan worden opgelost?

- ▶ Als er een **efficiënt algoritme** bestaat dat  $L$  beslist.

### Voorbeeld

$\text{PAL} = \{w \in \{0, 1\}^* \mid w \text{ is een palindroom}\}$  kan efficiënt worden opgelost.

Wanneer kunnen we zeggen dat  $L$  een (intrinsiek) moeilijk probleem is?

- ▶ Als er **geen** algoritme bestaat dat  $L$  op een efficiënte manier beslist.

### Voorbeeld

$\text{SAT} = \{w \in \{0, 1\}^* \mid w \text{ representeert een satisfiable proposionele formule}\}$  kan niet efficiënt worden opgelost. Meer details over dit probleem volgen later in de cursus.

We zien later in de cursus wat we bedoelen met “efficiënt oplossen.”

We kunnen ook kijken naar TM **die functies** berekenen. Hiervoor vervangen we  $q_{accept}$  and  $q_{reject}$  door een toestand  $q_{halt}$ . Definieer de output als de inhoud van de tape (tot aan de eerste  $\sqcup$ ) wanneer de TM stopt.

### Definition

Een TM **berekent een functie**  $f : \Sigma^* \rightarrow \Sigma^*$  als en slechts dan voor elk input woord  $w$ ,  $f(w)$  de output is van de TM.

### Voorbeeld

Binaire optelling, shifting, ...

Gelijkaardige definities bestaan om complexiteit van dergelijke TM te meten.

Echter, in deze cursus: vooral beslissingsproblemen.

- ▶ **Probleem:** Beslissingsproblemen (zit een woord in een taal?)
- ▶ **Algoritme:** Beslisser (TM die steeds stopt)
- ▶ **Complexiteit:** Asymptotische complexiteit (aantal stappen, gebruikte cellen op tape,...)

Hoewel we TMs en algoritmen identificeren (Church-Turing thesis), hangt de complexiteit af van **welk soort TM** men beschouwt...

Welke andere soorten TM bestaan er zoal? We bekijken er twee:

- ▶ TM met meerdere tapes; en
- ▶ Niet-deterministische TM (denk aan DFA versus NFA).

Deterministische Turing machines

Algoritmen & beslissers

Tijdscomplexiteit

Asymptotische analyse

Beslissingsproblemen

**Deterministische Turing machines met meerdere tapes**

Niet-Deterministische Turing machines

## Definition

Een deterministische TM met  $k$  tapes wordt gegeven door:  $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$  met:

- ▶  $Q$  is een eindige verzameling toestanden;
- ▶  $\Sigma$  is het input alfabet, met  $\vdash, \sqcup \notin \Sigma$ ;
- ▶  $\Gamma$  is het tape alfabet, met  $\Sigma \cup \{\vdash, \sqcup\} \subseteq \Gamma$ ;
- ▶  $q_0 \in Q$  is de begintoestand;
- ▶  $q_{accept}, q_{reject} \in Q$  zijn de aanvaardings- en verwerpingstoestanden; en
- ▶  $\delta$  is een partiële functie:

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{\leftarrow, \square, \rightarrow\}^k.$$

$\delta$  wordt de transitiefunctie genoemd.

We verwijzen naar dergelijke TMs als **multitape TMs**.

De machine heeft  $k$  **tapes**, oneindig doorlopend naar rechts.

- ▶ We gebruiken  $\vdash$  om de positie 0 op elke tape aan te duiden.
- ▶ Geen enkele transitie kan dit symbool overschrijven, of de lees/schrijfkop links ervan plaatsen (zoals voordien).

Stel,  $\Sigma$  is het input alfabet en  $\Gamma$  is het alfabet van de tapes.

- ▶ Het input woord  $w \in \Sigma^*$  van lengte  $n$  staat op posities  $1, \dots, n$  op de **eerste** tape.
- ▶ Op de overige posities  $(n + 1, n + 2, \dots)$  van de eerste tape staat  $\sqcup$ .
- ▶ De overige tapes bevatten alleen maar  $\sqcup$  in alle posities  $1, 2, 3, \dots$

De machine heeft **één lees/schrijfkop per tape**.

- ▶ In het begin, de machine is in toestand  $q_0$ , en alle koppen staan op positie 1 van hun tape.

Een **transitie** van de vorm

$$\delta(q, a_1, \dots, a_k) = (q', b_1, \dots, b_k, X_1, \dots, X_k)$$

met  $q, q' \in Q$ ,  $a_i, b_i \in \Gamma$  en  $X_i \in \{\leftarrow, \square, \rightarrow\}$  betekent:

- ▶ de machine is in toestand  $q$ ;
- ▶ de koppen  $1, 2, \dots, k$  lezen symbolen  $a_1, a_2, \dots, a_k$ ;
- ▶ de toestand verandert in  $q'$ ;
- ▶ de machine schrijft  $b_1$  op tape 1,  $b_2$  op tape 2, enz.; en
- ▶ de *ide* kop beweegt zoals aangegeven door symbool  $X_i$ .

**Configuraties** zijn net zoals voordien, maar nu met  $k$  tapes.

We hebben dus  $k$  **paartjes**  $u_i, v_i \in \Gamma^*$  nodig om de inhoud van deze tapes te beschrijven.

We gebruiken het symbol  $\#$  om de start van elke tape aan te duiden (in een configuratie).

Als de machine zich bevindt in toestand  $q$  and  $u_i v_i$  is de inhoud van tape  $i$ , dan is

$$\#u_1qv_1\#u_2qv_2\#\cdots\#u_kqv_k,$$

een configuratie waarin elke kop op het eerste symbool staat van  $v_i$ .

Voor de rest (configuratie naar configuratie, aanvaarden, verwerpen) blijft zoals bij TM met één tape.



Zoals voordien, de taal aanvaard door een multitape TM:

$$L(M) = \{w \in \Sigma^* \mid M \text{ aanvaardt } w\}.$$

(Let op:  $M$  is steeds een beslisser in deze cursus.)

Voor een multitape machine  $M$  met alfabet  $\Sigma$ :

- ▶ **Stap van  $M$ :** De uitvoering van één instructie van de transitiefunctie (verandering van configuratie)
- ▶  **$time_M(w)$ :** Aantal stappen dat de multitape TM  $M$  doet op input  $w \in \Sigma^*$
- ▶ **Worst case complexiteit van  $M$ :**

$$t_M(n) = \max\{ time_M(w) \mid w \in \Sigma^* \text{ en } |w| = n \}$$

- ▶ Een multitape TM  $M$  **loopt in tijd**  $\mathcal{O}(t(n))$  als  $t_M(n)$  **van orde**  $\mathcal{O}(t(n))$  is.

### Example

Er is een 2-tape TM  $M$  met tijdscomplexiteit  $\mathcal{O}(n)$  dewelke de taal  $L = \{w \in \{0, 1\}^* \mid w \text{ is een palindroom}\}$  aanvaardt.

$M =$  Op input woord  $w$ :

1. Allereerst, kopiëer de input van tape 1 naar tape 2.  $\mathcal{O}(n)$  **stappen**
2. Dan, zet de kop van de eerste tape op het eerste symbool van  $w$  (de 2de kop blijft staan op het laatste symbool).  $\mathcal{O}(n)$  **stappen**
3. Schuif de kop van tape 1 naar rechts, kop van tape 2 naar links.  $\mathcal{O}(1)$  **stappen**
4. Als er twee verschillende symbolen gelezen worden, verwerp de input, anders herhaal stap 3.  $\mathcal{O}(n)$  **keer**
5. Als de kop van tape 1 op het laatste symbool staat, aanvaard.

---

totaal: maximaal  $\mathcal{O}(n)$  **stappen**

Herinner: Een taal  $L$  wordt beslist door een TM  $M$  als  $L = L(M)$ .

Kunnen we **meer talen** beslissen met meerdere tapes?

### Stelling

*Als een taal  $L$  beslist wordt door een  $k$ -tape TM  $M$ , dan wordt deze taal ook beslist door een TM  $S$  met 1 tape.*

**Bewijs (informeel):** De  $k$  tapes van  $M$  worden eerst op de (enige) tape van  $S$  gekopieerd.

Hier gebruiken we  $\#$  om de inhoud van de verschillende tapes te onderscheiden (net zoals in de configuraties).

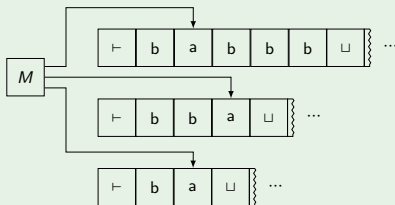
Om de positie van de koppen aan te duiden gebruikt  $S$  nieuwe symbolen op de tape.

Bijvoorbeeld, als kop staat over een symbool  $a$ , duiden we dit aan met  $\tilde{a}$ .

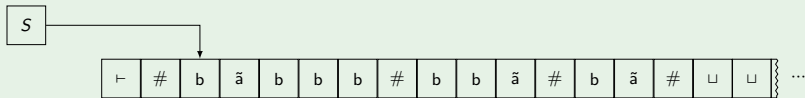
Maw,  $S$  heeft  $\tilde{a}$  in zijn tape alfabet, voor elke  $a$  in het tape alfabet van  $M$ .

Zie dit als **virtuele tapes en koppen**.

## Voorbeeld



Komt overeen met:



De 1-tape TM  $S$  werkt als volgt.

$S = \text{Op input } w = w_1 \cdots w_n$ :

1. **Simuleer** de initiële configuratie van  $M$  op  $w$ :

$$\vdash \# \tilde{w}_1 w_2 \cdots w_n \# \tilde{\sqcup} \# \tilde{\sqcup} \cdots \# \tilde{\sqcup} \#$$

2. Om een stap van  $M$  te simuleren scant  $S$  de tape om te kijken **waar elke kop** zich bevindt. Vervolgens maakt  $S$  een tweede scan van de tape en **updatet de tape** zoals beschreven door de transitiefunctie van  $M$ .
3. Als de virtuele kop naar *rechts* beweegt en op  $\#$  staat, wil dit zeggen dat  $M$  een kop beweegt naar een positie die nog niet bezocht was (and dus een  $\sqcup$  bevat). Dus,  $S$  schrijft een  $\sqcup$  en **schuift alle inhoud erna 1 positie naar rechts op**. Dit wordt ook een “rightshift” genoemd.  $\square$

Begin met TM  $M$  met  $k$  tapes:  $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ .

We definiëren nu een TM  $S = (Q', \Sigma', \Gamma', \delta', q'_0, q'_{accept}, q'_{reject})$  met 1 **tape**.

- ▶ Input alfabet:  $\Sigma' = \Sigma$ .
- ▶ Tape alfabet:  $\Gamma' = \Gamma \cup \{\#\} \cup \tilde{\Gamma}$ .
  - ▶ Symbool  $\#$ : om verschillende **tapes te onderscheiden**.
  - ▶  $\tilde{\Gamma} = \{\tilde{a} \mid a \in \Gamma\}$  om **kop posities** aan te duiden.
- ▶ Toestanden:

$$Q' = \{q'_0, q'_{accept}, q'_{reject}\} \cup (\text{fase} \times Q \times (\Gamma' \times \{\leftarrow, \square, \rightarrow\}))^k$$

- ▶ We gebruiken “fase” om toestanden te groeperen afhankelijk van wat de machine doet.
  - ▶ Wat deze fasen zijn wordt later duidelijk.
- ▶ Transitie die niet gespecificeerd zijn, leiden automatisch tot het verwerpen van de input.

**Doel:** beginconfiguratie aanmaken.

**Fase:**  $ini_1, \dots, ini_k$

Voor alle  $a \in \Gamma$ :

$$\delta'(q'_0, a) = ((ini_1, q_0, \tilde{a}, \rightarrow, \underbrace{\sqcup, \rightarrow, \dots, \sqcup, \rightarrow}_{k-1 \text{ keer}}), \#, \rightarrow)$$

- ▶ We onthouden wat we gelezen hebben, schrijven  $\#$  en bewegen naar rechts.
- ▶ We onthouden welke tape we “virtueel” bezoeken met de fase  $ini_1, \dots, ini_k$ .
- ▶ Eerste symbool (hier  $a$ ) onthouden we in de toestand als  $\tilde{a}$ , om later de kop positie te kennen en die op volgende positie te zetten.



Voor alle  $a \in \Gamma' \setminus \{\sqcup\}$  and  $b \in \Gamma \setminus \{\sqcup\}$ :

$$\delta'((ini_1, q_0, a, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), b) = ((ini_1, q_0, b, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), a, \rightarrow)$$

- ▶ Wanneer het huidige symbool ( $b$ ) geen blank is, onthouden wat we gelezen hebben (hier  $b$ ), schrijven dat symbool in de toestand, schrijven het vorige symbool op de tape, en bewegen naar rechts.
- ▶ Herinner, eerst gelezen symbool, stel  $a$  wordt  $\tilde{a}$  op tape.

Voor  $b = \sqcup$  of  $a = \sqcup$ :

$$\delta'((ini_1, q_0, a, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \sqcup) = ((ini_1, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), a, \rightarrow)$$

$$\delta'((ini_1, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \sqcup) = ((ini_2, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \#, \rightarrow)$$

- ▶ Gelezen symbol (voor tape 1) is  $\sqcup$ , we schrijven  $\#$ , en beginnen nu aan tape 2.
- ▶ We moeten  $\tilde{\sqcup}\#$  schrijven voor tape 2, 3, ...,  $k$ .

$$\begin{aligned}
 \delta'((ini_2, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \sqcup) &= ((ini_2, q_0, \#, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \tilde{\sqcup}, \rightarrow) \\
 \delta'((ini_2, q_0, \#, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \sqcup) &= ((ini_3, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \#, \rightarrow) \\
 \delta'((ini_3, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \sqcup) &= ((ini_3, q_0, \#, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \tilde{\sqcup}, \rightarrow) \\
 \delta'((ini_3, q_0, \#, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \sqcup) &= ((ini_4, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \#, \rightarrow) \\
 &\vdots \\
 \delta'((ini_k, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \sqcup) &= ((ini_k, q_0, \#, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \tilde{\sqcup}, \rightarrow) \\
 \delta'((ini_k, q_0, \#, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \sqcup) &= ((back, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \#, \square)
 \end{aligned}$$

- ▶ We schrijven  $\tilde{\sqcup}\#$  voor tape  $i = 2, 3, \dots, k$ .
- ▶ Op het eind, gaan we over naar een andere fase: “back”.

De initiële configuratie van  $M$  staat nu op de tape van  $S$ .

**Doel:** We gaan terug naar het begin en zetten kop op eerste niet  $\#$ -symbool.

**Fase:** back, step

Voor alle  $a \in \Gamma' \setminus \{\vdash\}$ :

$$\delta'((\text{back}, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), a) = ((\text{back}, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), a, \leftarrow)$$

voor  $\vdash$ :

$$\delta'((\text{back}, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \vdash) = ((\text{step}, q_0, \vdash, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \vdash, \rightarrow)$$

$$\delta'((\text{step}, q_0, \vdash, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \#) = ((\text{read}_1, q_0, \sqcup, \rightarrow, \sqcup, \rightarrow, \dots, \sqcup, \rightarrow), \#, \rightarrow)$$

- ▶ Het gebruik van  $\vdash$  komt hier handig van pas. Merk op dat het eerste symbool na  $\vdash$  het symbool  $\#$  is.
- ▶ We gaan we over naar een andere fase: “read”.

De initiële configuratie van  $M$  staat nu op de tape van  $S$  en de kop van  $S$  staat op het eerste niet  $\#$  symbool.

**Doel:** We lezen de virtuele input tape(s), kijken waar de koppen staan, en bewaren de  $\sim$ -symbolen in de toestanden.

**Fase:**  $\text{read}_1, \dots, \text{read}_k$

Hier,  $q \in Q$  is een toestand van  $M$ .

Voor de letters onder de kop:

$\delta'((\text{read}_i, q, a_1, d_1, \dots, a_k, d_k), \tilde{a})$  is gedefinieerd als  
 $((\text{read}_i, q, a_1, d_1, \dots, a_{i-1}, d_{i-1}, \textcolor{red}{a}, d_i, a_{i+1}, d_{i+1}, \dots, a_k, d_k), \tilde{a}, \rightarrow)$

En voor de andere letters, behalve  $\#$ :

$\delta'((\text{read}_i, q, a_1, d_1, \dots, a_k, d_k), a)$  is gedefinieerd als  
 $((\text{read}_i, q, a_1, d_1, \dots, a_k, d_k), a, \rightarrow)$

Tenslotte, voor  $i = 1, \dots, k - 1$  en symbool  $\#$ :

$\delta'((\text{read}_i, q, a_1, d_1, \dots, a_k, d_k), \#)$  is gedefinieerd als

$$((\text{read}_{i+1}, q, a_1, d_1, \dots, a_k, d_k), \#, \rightarrow)$$

en voor het laatste  $\#$  symbool:  $\delta'((\text{read}_k, q, a_1, d_1, \dots, a_k, d_k), \#)$  is

$$((\text{backread}, q, a_1, d_1, \dots, a_k, d_k), \#, \leftarrow)$$

We hebben nu

- ▶ de volledige tape gelezen; en
- ▶ symbolen die de  $k$  koppen lezen zijn opgeslagen in de toestanden.
- ▶ We gaan over naar een andere phase: "backread".

All informatie om een transitie van  $M$  te simuleren zit nu in de toestand van  $S$ !

**Doel:** Na het vinden van letters onder koppen, gaan we terug naar het begin en zetten de kop op eerste niet  $\#$  letter.

**Fase:** backread, step'

$$\delta'((\text{backread}, q, a_1, d_1, \dots, a_k, d_k), a) = ((\text{backread}, q, a_1, d_1, \dots, a_k, d_k), a, \leftarrow)$$

$$\delta'((\text{backread}, q, a_1, d_1, \dots, a_k, d_k), \vdash) = ((\text{step}', q, a_1, d_1, \dots, a_k, d_k), \vdash, \rightarrow)$$

$$\delta'((\text{step}', q, a_1, d_1, \dots, a_k, d_k), \#) = ((\text{trans}, q, a_1, d_1, \dots, a_k, d_k), \#, \rightarrow)$$

- Ok, terug op eerste positie.
- Over naar andere fase: "trans".

De kop van  $S$  staat klaar om de transitie  $\delta(q, (a_1, \dots, a_k))$  van  $M$  te maken.

**Doel:** Simuleer de transitie van  $M$ .

**Fase:**  $\text{trans}, \text{trans}_1, \dots, \text{trans}_k, \text{movenot}_1, \dots, \text{movenot}_k, \text{moveleft}_1, \dots, \text{moveleft}_k, \text{moveright}_1, \dots, \text{moveright}_k$ .

Als  $\delta(q, (a_1, \dots, a_k)) = (q', (b_1, \dots, b_k), (e_1, \dots, e_k))$  met  $q' \in Q$ ,  $b_i \in \Gamma$  and  $e_i \in \{\leftarrow, \square, \rightarrow\}$  dan

$$\delta'((\text{trans}, q, a_1, d_1, \dots, a_k, d_k), a) = ((\text{trans}_1, q', b_1, e_1, \dots, b_k, e_k), a, \square)$$

In de toestand staat nu de instructie om de transitie te maken.

Voor  $a \neq \#$ ,  $a$  is geen “tilde” symbool: ga naar rechts:

$$\delta'((\text{trans}_i, q', b_1, e_1, \dots, b_k, e_k), a) = ((\text{trans}_i, q', b_1, e_1, \dots, b_k, e_k), a, \rightarrow)$$

Voor  $a = \#$ ,  $i \neq k$ , ga naar volgende “virtuele” tape:

$$\delta'((\text{trans}_i, q', b_1, e_1, \dots, b_k, e_k), \#) = ((\text{trans}_{i+1}, q', b_1, e_1, \dots, b_k, e_k), \#, \rightarrow)$$

Voor  $a = \#$ ,  $i = k$ , ga terug naar begin en start opnieuw met “read” fase:

$$\delta'((\text{trans}_k, q', b_1, e_1, \dots, b_k, e_k), \#) = ((\text{back}, q', b_1, e_1, \dots, b_k, e_k), \#, \square)$$

$$\delta'((\text{back}, q', b_1, e_1, \dots, b_k, e_k), a) = ((\text{back}, q', b_1, e_1, \dots, b_k, e_k), a, \leftarrow)$$

$$\delta'((\text{back}, q', b_1, e_1, \dots, b_k, e_k), \vdash) = ((\text{step}, q', b_1, e_1, \dots, b_k, e_k), \vdash, \rightarrow)$$

$$\delta'((\text{step}, q', b_1, e_1, \dots, b_k, e_k), \#) = ((\text{read}_1, q', b_1, e_1, \dots, b_k, e_k), \#, \rightarrow)$$



Voor  $\tilde{a}$  definiëren we  $\delta'((\text{trans}_i, q', b_1, e_1, \dots, b_k, e_k), \tilde{a})$  als

$$\begin{cases} ((\text{movenot}_i, q', b_1, e_1, \dots, b_k, e_k), \tilde{b}_i, \square) & \text{als } e_i = \square \\ ((\text{moveleft}_i, q', b_1, e_1, \dots, b_k, e_k), b_i, \leftarrow) & \text{als } e_i = \leftarrow \\ ((\text{moveright}_i, q', b_1, e_1, \dots, b_k, e_k), b_i, \rightarrow) & \text{als } e_i = \rightarrow \end{cases}$$

waarbij

$$\delta'((\text{movenot}_i, q', b_1, e_1, \dots, b_k, e_k), \tilde{b}_i) = ((\text{trans}_i, q', b_1, e_1, \dots, b_k, e_k), \tilde{b}_i, \rightarrow)$$

en

$$\delta'((\text{moveleft}_i, q', b_1, e_1, \dots, b_k, e_k), a) = ((\text{trans}_i, q', b_1, e_1, \dots, b_k, e_k), \tilde{a}, \rightarrow)$$

en als  $a \neq \#$ :

$$\delta'((\text{moveright}_i, q', b_1, e_1, \dots, b_k, e_k), a) = ((\text{trans}_i, q', b_1, e_1, \dots, b_k, e_k), \tilde{a}, \rightarrow).$$

Wanneer in fase  $\text{moveright}_i$  en het eindsymbool  $\#$  gelezen wordt, moeten we alles rechts daarvan 1 positie opschuiven en een  $\tilde{\#}$  schrijven op de vrijgekomen positie.

$$\delta'((\text{moveright}_i, q', b_1, e_1, \dots, b_k, e_k), \#) = ((\text{shiftright}\#_i, q', b_1, e_1, \dots, b_k, e_k), \tilde{\#}, \rightarrow)$$

$$\delta'((\text{shiftright}_i, q', b_1, e_1, \dots, b_k, e_k), b) = ((\text{shiftright}_i, q', b_1, e_1, \dots, b_k, e_k), a, \rightarrow)$$

$$\delta'((\text{shiftright}\#_i, q', b_1, e_1, \dots, b_k, e_k), a) = ((\text{shiftright}_{i+1}, q', b_1, e_1, \dots, b_k, e_k), \#, \rightarrow)$$

$$\delta'((\text{shiftright}\#_k, q', b_1, e_1, \dots, b_k, e_k), \sqcup) = ((\text{moveback}\tilde{\#}, q', b_1, e_1, \dots, b_k, e_k), \sqcup, \leftarrow)$$

- We bewaren de gelezen letters in de fase om die nadien te kunnen schrijven op de volgende positie.
- Met behulp van  $\text{moveback}\tilde{\#}$  terug naar het enige  $\tilde{\#}$ -symbool gaan,  $\tilde{\#}$  schrijven, naar rechts gaan en  $\text{trans}_{i+1}$  verder zetten.

Zodra we naar een aanvaardingstoestand gaan (bijvoorbeeld) in het begin van de transitie phase, simpelweg aanvaard/verwerp.

$$\delta'((\text{trans}, q_{\text{accept}}, a_1, d_1, \dots, a_k, d_k), a) = (q'_{\text{accept}}, a, \square)$$

$$\delta'((\text{trans}, q_{\text{reject}}, a_1, d_1, \dots, a_k, d_k), a) = (q'_{\text{reject}}, a, \square).$$

- Een verder formeel bewijs is nog nodig om aan te tonen dat  $L(M) = L(S)$ .
- Typisch per inductie op het aantal stappen van de TMs.

Is de tijdscomplexiteit kleiner wanneer we meer tapes mogen gebruiken?

### Stelling

*Als een taal  $L$  beslist wordt door een TM  $M$  met  $k$  tapes ( $k \geq 2$ ) in tijd  $\mathcal{O}(t(n))$ , met  $t(n) \geq n$ , dan wordt  $L$  beslist door een TM  $S$  met 1 tape in tijd  $\mathcal{O}(t(n)^2)$ .*

Dit volgt uit de simulatie van een multitape TM  $M$  door een TM  $S$  met één tape.

Initialisatie,  $S$  kopieert inhoud op 1 tape.

- ▶ Input tape bevat  $\mathcal{O}(n)$  symbolen.
- ▶  $S$  doet  $\mathcal{O}(n)$  stappen tijdens initialisatie.

Voor elke stap van  $M$  gaat  $S$  **twee keer** over de tape.

- ▶ Eén keer om de koppen te vinden:
  - ▶ Aangezien  $M$   $\mathcal{O}(t(n))$  stappen zet, zijn er slechts  $\mathcal{O}(t(n))$  posities op de tape van  $M$  van belang.
  - ▶ Dus,  $S$  doet  $\mathcal{O}(t(n))$  stappen om alles te lezen.
- ▶ Eén keer om de koppen te verzetten en symbolen te schrijven.
  - ▶ Dit kan leiden tot maximaal  $k$  “rightshifts”, in elk van deze doet  $S$  maximaal  $\mathcal{O}(t(n))$  stappen.
  - ▶ Nu,  $k$  is een vast getal (aantal tapes), dus  $\mathcal{O}(k \cdot t(n)) = \mathcal{O}(t(n))$ .

In totaal,  $\mathcal{O}(n) + \mathcal{O}(t(n)) \cdot \mathcal{O}(t(n)) = \mathcal{O}(t(n)^2)$  stappen (herinner: we nemen aan dat  $t(n) \geq n$ ).

### Stelling

*Als een taal  $L$  beslist wordt door een TM  $M$  met  $k$  tapes ( $k \geq 2$ ) in tijd  $\mathcal{O}(t(n))$ , met  $t(n) \geq n$ , dan wordt  $L$  beslist door een TM  $S$  met 1 tape in tijd  $\mathcal{O}(t(n)^2)$ .*

- Kunnen we dit verschil in tijdscomplexiteit tussen  $M$  en  $S$  vermijden? Nee!

Neem de taal  $L = \{w \in \{0, 1, \#\}^* \mid w \text{ is een palindroom}\}$ .

- ▶  $L$  wordt beslist door een 2-tape TM in tijd  $\mathcal{O}(n)$
- ▶  $L$  wordt beslist door een 1-tape TM in tijd  $\mathcal{O}(n^2)$ .

Kan  $L$  worden beslist in lineaire tijd door TM met 1 tape?

### Propositie

*Zij  $M$  een TM met 1 tape die  $L$  (palindromen) aanvaardt. Dan heeft  $M$  tijdscomplexiteit  $\Omega(n^2)$ .*

**Bewijs:** Stel,  $L = L(M)$  met  $M$  een TM met 1 tape.

Laat  $Q$  de verzameling van toestanden van  $M$  zijn.

Merk op, we kunnen veronderstellen dat  $M$  het volledige input woord leest (anders kan die zowieso niet beslissen of een woord een palindroom is).

Intuïtief gezien gaat  $M$  heel de tijd van een positie  $i$  op de tape naar positie  $n - i$  moeten gaan om de symbolen te vergelijken.

Omdat de toestanden in een TM slechts een eindige hoeveelheid informatie kunnen bevatten, gaat het dergelijke heen-en-weer bewegingen vaak moeten doen.

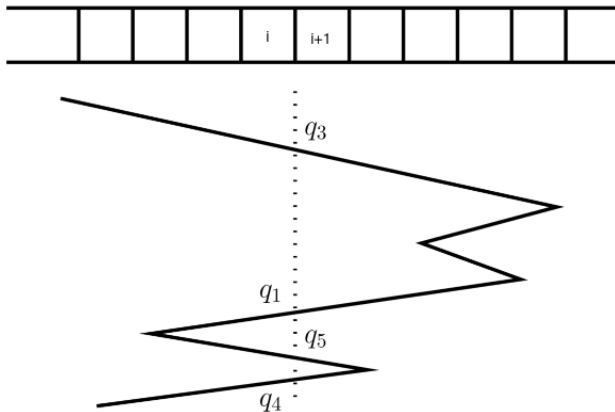
Heen-en-weer: **Crossing sequences**

### Definition

Voor een input woord  $x$  en positie  $i$  op de tape definiëren we  $C_i(x)$  als de lijst van toestanden in  $Q$  waarin  $M$  zich bevindt als deze beweegt tussen positie  $i$  and  $i + 1$  op input  $x$ .



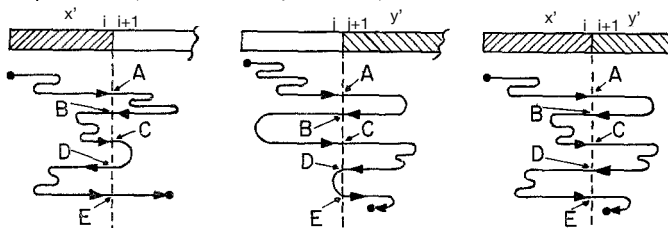
Figuur ter verheldering:



## Lemma

Voor elke twee woorden  $x$  en  $y$  en posities  $i$  en  $j$  geldt het volgende: als  $C_i(x) = C_j(y)$  dan ook  $C_i(x) = C_i(x'y') = C_j(y)$  met

- ▶  $x'$  eerste  $i$  letters van  $x$
- ▶  $y'$  laatste  $|y| - j$  letters van  $y$ .



### Lemma

*Als  $C_i(x) = C_j(y)$  en  $x$  en  $y$  beide aanvaard worden door  $M$  dan zal ook  $x'y'$  aanvaard worden door  $M$ .*

**Bewijs:** Omdat  $M$  het woord  $x$  aanvaardt na alle toestanden in  $C_i(x)$  doorlopen te hebben (met de bijhorende crossings van positie  $i$  naar  $i + 1$ ) zal  $M$   $x$  aanvaarden met de kop ofwel links (eerste  $i$  posities) ofwel rechts (laatste  $|x| - i$  posities) in  $x$ . Aangezien  $C_i(x) = C_j(y)$  zal  $M$  dus ook  $y$  aanvaarden met de kop ofwel op de eerste  $j$  posities van  $y$  ofwel de laatste  $|y| - j$  posities van  $y$ .

Aangezien  $C_i(x) = C_i(x'y') = C_j(y)$  en de “run” van  $M$  links van  $i$  en rechts van  $j$  slechts afhangt van de toestanden in de crossing sequences, zal deze ook  $x'y'$  aanvaarden.  $\square$

We beschouwen nu speciale palindromen waarvoor we controle hebben op het **minimum aantal crossings**. (Herinner: we willen een benedengrens aantonen.)

Voor een woord  $w \in \{0, 1, \#\}^*$ , noteren we met  $w^r$  het woord  $w$  maar dan opgeschreven van achter naar voor.

Definiëer nu de taal  $L_n$  ( $n > 0$  and  $n$  is deelbaar door 4):

$$L_n := \{w\#^{\frac{n}{2}}w^r \mid w \in \{0, 1\}^{\frac{n}{4}}\}.$$

Het is duidelijk dat  $L_n \subseteq L$ .

Verzamel nu alle crossing sequences voor een woord  $w$  voor welbepaalde posities  $i$ :

$$C(w) := \{C_i(w) \mid \frac{n}{4} < i \leq \frac{3n}{4}\}.$$

### Lemma

Als  $w_1, w_2 \in L_n$  en  $w_1 \neq w_2$ , dan is  $C(w_1) \cap C(w_2) = \emptyset$ .

**Bewijs:** Veronderstel dat het lemma niet waar is. Dit will zeggen dat er posities  $i$  and  $j$  moeten bestaan,  $i, j \in \{\frac{n}{4} + 1, \dots, \frac{3n}{4}\}$  zodat  $C_i(w_1) = C_j(w_2)$ .

Neem nu voor  $u_1$  en  $v_2$  de woorden bestaande uit de eerste  $i$  symbolen van  $w_1$ , en de laatste  $n - j$  symbolen van  $w_2$ , respectievelijk.

Aangezien  $C_i(w_1) = C_j(w_2)$ , hebben we dus dat  $u_1 v_2$  aanvaard wordt door  $M$ . (dat hebben we zonet bewezen).

Maar  $u_1 v_2$  is geen palindroom (omdat  $w_1 \neq w_2$ )!



Dit geeft dus een manier om het aantal verschillende woorden in  $L_n$  in verband te brengen met het aantal crossing sequences.

Voor  $w \in L_n$ , laat  $s_w$  de **kortste lijst** in  $C(w)$  zijn. Definiëer:

$$\blacktriangleright S_n = \{s_w \mid w \in L_n\}$$

Merk op, elke  $s_w$  is van de vorm  $C_i(w)$  voor een  $i \in (\frac{n}{4}, \frac{3n}{4}]$ .

We weten van het vorige lemma dat  $s_{w_1} \neq s_{w_2}$  als  $w_1 \neq w_2$ . Inderdaad  $C(w_1) \cap C(w_2)$  is leeg!

Met andere woorden, de afbeelding  $f : L_n \rightarrow S_n : w \mapsto f(w) = s_w$  is een *bijjectie*:

- $\blacktriangleright$  Dus,  $|S_n| = |L_n| = 2^{\frac{n}{4}}$  aangezien er  $2^{\frac{n}{4}}$  verschillende woorden zijn in  $L_n$ .

Neem voor  $m$  de lengte van de **langste lijst** in  $S_n$ .

- $\blacktriangleright$  Het aantal lijsten van lengte **maximaal**  $m$  is:

$$\sum_{i=0}^m |Q|^i = \frac{|Q|^{m+1} - 1}{|Q| - 1}$$

omdat  $|Q|^i$  een bovengrens is op aantal lijsten van lengte  $i$ .

We mogen dus concluderen dat :  $\frac{|Q|^{m+1}-1}{|Q|-1} \geq 2^{\frac{n}{4}}$ .

- Waarom? (Lemma  $\Rightarrow$  all kortste lijsten zijn verschillend)

Dit betekent dat

$$\log_2(|Q|^{m+1} - 1) - \log_2(|Q| - 1) \geq n/4.$$

Nu,  $|Q| \geq 3$  (omdat  $Q$  zeker  $q_0$ ,  $q_{accept}$ ,  $q_{reject}$  bevat) en  $\log_2(x) \geq 0$  voor  $x \geq 1$ .

Ook hebben we dat  $\log_2(x) \leq \log_2(y)$  wanneer  $1 \leq x \leq y$  (monotone functie).

Dus

$$\log_2(|Q|^{m+1} - 1) - \log_2(|Q| - 1) \leq \log_2(|Q|^{m+1}) = (m+1) \log_2(|Q|).$$

en dus

$$(m+1) \log(|Q|) \geq \frac{n}{4}.$$

En dus  $m = \Omega(n)$ .

En dus  $m = \Omega(n)$ .

- ▶ Er is dus een  $w_0 \in L_n$  met  $|s_{w_0}|$  van grootte  $\Omega(n)$ .

Als gevolg: Alle lijsten in  $C(w_0)$  zijn van lengte  $\Omega(n)$  (omdat de kortste zo is).

**Conclusie:** Op input  $w_0$  loop de machine  $M$  in tijd  $\Omega(n^2)$ .

- ▶ Inderdaad,  $M$  moet  $\frac{n}{2}$  lijsten van toestanden van lengte  $\Omega(n)$  genereren (één voor elke kruising tussen positie  $\frac{n}{4}$  and  $\frac{3n}{4}$ )
- ▶ Dit kan alleen maar door  $\Omega(n^2)$  stappen te zetten. □

Ook al aanvaarden 1-tape en  $k$ -tape TMs dezelfde talen kan de tijdscomplexiteit verschillen.



Deterministische Turing machines

Algoritmen & beslissers

Tijdscomplexiteit

Asymptotische analyse

Beslissingsproblemen

Deterministische Turing machines met meerdere tapes

Niet-Deterministische Turing machines

### Definition

Een **niet-deterministische TM (NTM)** wordt gegeven door  $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ , met daarin:

- ▶  $Q$  is een eindige verzameling van toestanden
- ▶  $\Sigma$  is het input alfabet, met  $\vdash, \sqcup \notin \Sigma$
- ▶  $\Gamma$  is het tape alfabet, met  $\Sigma \cup \{\vdash, \sqcup\} \subseteq \Gamma$
- ▶  $q_0 \in Q$  is de begintoestand
- ▶  $q_{accept}, q_{reject} \in Q$  zijn de aanvaardings- en verwerpingstoestanden
- ▶  $\delta$  is een **relatie (!)**:

$$\delta \subseteq Q \times \Gamma \times Q \times \Gamma \times \{\leftarrow, \square, \rightarrow\}$$

Begintoestand, configuratie en tape inhoud zijn zoals voordien.

Echter, we hebben nu een **keuze** hoe van de ene naar de andere configuratie te gaan.

Veronderstel dat de kop het symbool  $a$  leest en dat de machine zich in toestand  $q$  bevindt.

Bijvoorbeeld, stel dat  $(q, a, q', b, \square)$  en  $(q, a, q'', a, \rightarrow)$  beiden in  $\delta$  zitten, dan:

- ▶ kunnen we **ofwel**  $b$  schrijven, de toestand veranderen in  $q'$  en de kop op dezelfde positie houden,
- ▶ **ofwel** behouden we  $a$ , veranderen de toestand in  $q''$  en maken we een stap naar rechts.

We gaan van de ene configuratie  $C_1$  naar een andere configuratie  $C_2$  wanneer, net zoals voordien:

Veronderstel,  $a, b, c \in \Gamma$ , en  $u, v \in \Gamma^*$  en  $q_i, q_j \in Q$

$\vdash uaq_i bv$  leidt tot  $\vdash uq_j acv$ , wanneer  $(q_i, b, q_j, c, \leftarrow) \in \delta$ ;

$\vdash uaq_i bv$  leidt tot  $\vdash uacq_j v$ , wanneer  $(q_i, b, q_j, c, \rightarrow) \in \delta$ ;

$\vdash uaq_i bv$  leidt tot  $\vdash uaq_j cv$ , wanneer  $(q_i, b, q_j, c, \square) \in \delta$ .

Een niet-deterministische TM  $M$  **aanvaardt een input woord**  $w$  als:

Er een reeks van configuraties  $C_1, C_2, \dots, C_k$  bestaat zodat:

1.  $C_1$  is de beginconfiguratie van  $M$  op input  $w$ ;
2.  $C_i$  leidt tot  $C_{i+1}$ ; en
3.  $C_k$  is een aanvaardingsconfiguratie.

Net zoals voordien:

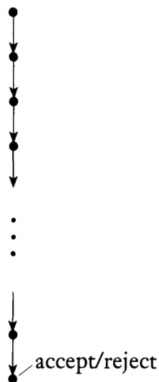
### Definition

De **taal aanvaard** door een niet-deterministische Turing machine  $M$ :

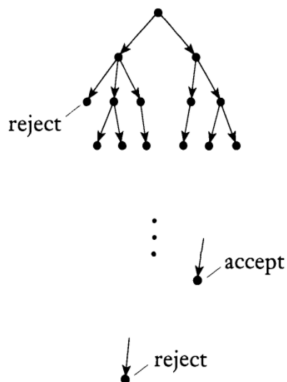
$$L(M) = \{w \in \Sigma^* \mid M \text{ aanvaardt } w\}.$$

We hebben slechts **één aanvaardingsreeks** van configuraties nodig om een taal te beslissen!

Deterministic



Nondeterministic



Dit wordt ook de **berekeningsboom** van een NTM genoemd.

Herinner: We willen weten of een woord al dan niet in een taal zit.

Door het niet-determinisme kunnen sommige “runs” aanvaarden, maar we weten niet wanneer dit gebeurt.

### Definition

Een niet-deterministische TM is een **beslisser** als elke run van deze machine stopt op elke input.

Zoals voordien:

**Niet-deterministisch algoritme** = een niet-deterministische TM die een beslisser is.

Stel  $M$  is een NTM die een beslisser is:

- ▶ **Stap van  $M$ :** Het uitvoeren van één instructie van de transitie relatie (1 “verandering” van configuratie).
- ▶  $time_M(w)$ : Maximum aantal stappen van  $M$  van **elke** run op input  $w$ .

## Definition

De **tijdscomplexiteit** van een NTM  $M$  is gedefinieerd als de functie  $t_M$ :

$$t_M(n) = \max\{ time_M(w) \mid w \in \Sigma^* \text{ en } |w| = n \}.$$

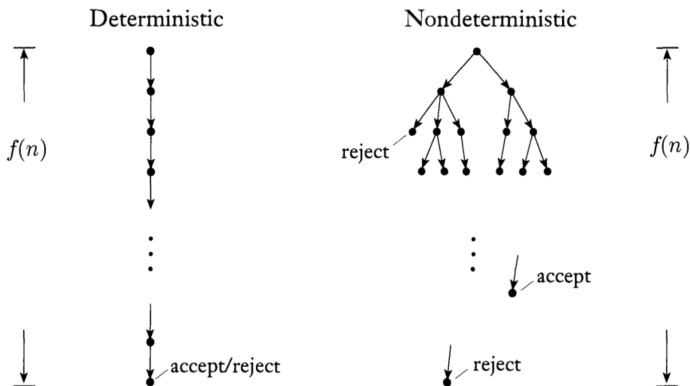
De tijd van een NTM  $M$  is dus een functie  $t_M : \mathbb{N} \rightarrow \mathbb{N}$ , zodat  $t_M(n)$  het maximaal aantal stappen is dat  $M$  doet in elk mogelijke run op een input van lengte  $n$ .

- ▶ Een NTM  $M$  **loopt in tijd**  $\mathcal{O}(t(n))$  als  $t_M(n)$  **van orde**  $\mathcal{O}(t(n))$  is.



## Hoe dit visualiseren?

Voor woorden van lengte  $n$  hebben we:



Hier noteert  $f(n)$  de functie  $t_M(n)$ .

Kunnen we meer talen beslissen door middel van NTMs?

Nee, maar wel op een (mogelijk) meer efficiënte manier:

### Theorem

*Stel,  $t(n)$  is een functie zodat  $t(n) \geq n$ . Dan, elke niet-deterministische TM  $N$  die loopt in tijd  $\mathcal{O}(t(n))$  is equivalent met een deterministische TM  $D$  die loopt in tijd  $2^{\mathcal{O}(t(n))}$ .*

**Bewijs:** Idee is om de **berekeningsboom** van NTM  $N$  te inspecteren d.m.v. een deterministische TM  $D$ .

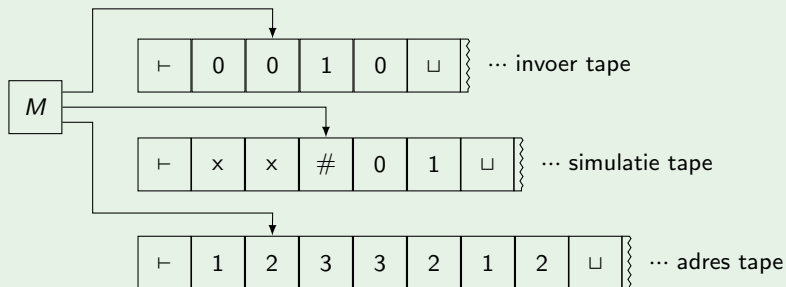
Wanneer een tak in deze boom (run van de machine  $N$ ) de input aanvaard, doen we dat ook. Zoniet, blijven we andere takken inspecteren.

We simuleren  $N$  met een **3-tape** deterministische machine  $D$ :

- ▶ Tape 1 bevat steeds de input  $w$ ;
- ▶ Tape 2 simuleert 1 run van  $N$  op input  $w$ ; en
- ▶ Tape 3 houdt bij welke tak (run) we inspecteren.

Nadien zetten we  $D$  om in een deterministische TM met **één tape**.

## Voorbeeld



We exploreren de berekeningsboom d.m.v. **Breadth-First-Search** (BFS)

- ▶ Depth-First-Search werkt niet omdat we zo een oneindige run kunnen belanden, in geval  $N$  geen beslisser is.

Stel dat  $b$  het maximaal aantal keuzes is dat we kunnen maken a.h.v.  $N$ 's transitierelatie  $\delta$ .

Op tape 3 **enumereren** we (lexicografisch) woorden over alfabet  $\{1, 2, \dots, b\}$ , eerst woorden van lengte 1, dan lengte 2, enzovoorts (BFS).

Deze specificeren waar te gaan in de boom.

- ▶  $1111\dots$  = steeds eerste keuze,  $bbbb\dots$  = steeds laatste keuze.

(Merk op: sommig adressen op tape 3 hebben geen betekenis)

Op Tape 2 simuleren we de gekozen tak in de boom (run van algoritme  $N$  op input).

$D$  werkt als volgt:

1. Start met  $w$  op tape 1. De andere tapes zijn leeg.
2.  $s = 0$  en genereer leeg adres op tape 3.
3. Kopiëer tape 1 naar tape 2.
4. Simuleer  $N$ 's run a.h.v. de keuzes op tape 3. Als deze run aanvaardt, aanvaard. Anders ga naar stap 5.
5. Vervang het woord op tape 3 met het volgende woord van lengte  $s$  (lexicografisch). Als alle woorden van lengte  $s$  zijn gegenereerd, zet  $s = s + 1$  en genereer “eerste” woord op tape 3 van lengte  $s$ . Ga naar stap 3.

Merk op: Als  $N$  een beslisser is dan kunnen we  $D$  ook tot een beslisser maken:

- ▶ De berekeningsboom van  $N$  is eindig
- ▶ Laat  $D$  de input verwerpen als de ganse boom beschouwd is.

Wat is de complexiteit?

Kopiëren neemt  $\mathcal{O}(n)$  tijd.

We volgen een tak, dit neemt  $\mathcal{O}(t(n))$  tijd. Dus, asymptotisch neemt dit tijd  $ct(n)$  for een zekere  $c \in \mathbb{N}$ .

Het aantal adressen (takken) is maximaal

$$\sum_{i=1}^{ct(n)} b^i = \frac{1 - b^{ct(n)+1}}{1 - b} - 1 = b \left( \frac{1 - b^{ct(n)}}{1 - b} \right) \leq \frac{b}{b-1} b^{ct(n)} \leq 2b^{ct(n)} = \mathcal{O}(b^{t(n)})$$

omdat  $b \geq 2$ .

In totaal,

$$\mathcal{O}((n + t(n))b^{t(n)}) = \mathcal{O}(t(n)b^{t(n)}) = \mathcal{O}(2^{\log_2(t(n)) + \log_2(b)t(n)}) = 2^{\mathcal{O}(t(n))}$$

stappen, omdat  $t(n) \geq n \geq \log_2(n)$ .

Na, omzetten naar TM met één tape:

$$(2^{\mathcal{O}(t(n))})^2 = 2^{\mathcal{O}(2t(n))} = 2^{\mathcal{O}(t(n))}$$

stappen.

### Theorem

*Stel,  $t(n)$  is een functie zodat  $t(n) \geq n$ . Dan, elke niet-deterministische TM  $N$  die loopt in tijd  $\mathcal{O}(t(n))$  is equivalent met een deterministische TM  $D$  die loopt in tijd  $2^{\mathcal{O}(t(n))}$ .*

Het is niet geweten of deze exponentiële “blowup” vermeden kan worden!!

TM met 1 tape, meerdere tapes al dan niet met niet-determinisme aanvaarden **dezelfde** talen maar met mogelijk **verschillende efficiëntie**.



- ▶ Formalisme van Turing machines (Sipser 3.1)
- ▶ Varianten van TM (Sipser 3.2)
- ▶ Beslissers en algoritmen (Sipser 3.3)
- ▶ Worst-case analysis (Big-O..., Sipser 7.1)
- ▶ Simulatie van  $k$ -tape TM door 1-tape TM, niet-deterministische TM door 1-tape deterministische TM en bijbehorend verschil in complexiteit (Sipser 7.1)
  - ▶ Crossing sequences: begrijp idee van het bewijs en weet dat palindromen niet in linear tijd met 1-tape TM kunnen aanvaard worden.